



泛在计算与智能系统研究中心

实现篇

第十一章 数据库恢复

主讲：张浩





研究动机

- 数据库恢复机制用来进行事务终止后的善后处理和系统故障恢复处理，确保系统发生故障以后，数据库能够恢复到正确的状态。





- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
- 检查点
- 缓冲技术





- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
- 检查点
- 缓冲技术





- 故障可能会破坏数据库的一致性
 - 已提交的事务对数据库的修改未全部持久化到硬盘
 - 破坏事务的持久性
 - 已终止的事务对数据库的修改已部分持久化到硬盘
 - 破坏事务的原子性





数据库恢复的必要性



- 数据库系统的数据库恢复机制的目的有两个：
 - 第一个目的是保证**事务的原子性**，即确保一个事务被交付运行之后，要么该事务中的所有数据库操作都被成功地完成而且这些操作的结果被永久地存储到数据库中，要么这个事务对数据库没有任何影响。
 - 第二个目的是当系统发生故障以后，数据库能够恢复到**正确状态**。





数据库恢复的必要性

• 故障分类

— 事务故障

- 逻辑故障：事务由于某些内部条件而无法继续正常执行，如非法输入、找不到数据、溢出或超出资源限制
- 系统错误：系统进入一种不良状态，如死锁，结果事务无法继续正常执行，但该事务可以在以后的某个时间重新执行

— 系统崩溃

- 软件故障：如DBMS实现的bug导致
- 硬件故障，或断电，导致易失性存储器内容的丢失，并使得事务处理停止，而非易失性存储器仍完好无损

— 磁盘故障

- 在数据传送操作过程中由于磁头损坏或故障造成磁盘块上的内容丢失。其他盘上的备份可以恢复这种故障





数据库恢复的必要性

• 故障恢复与处理

- 首先，确定用于存储数据的设备的故障方式
- 其次，必须考虑这些故障方式对数据库的影响
- 然后，提出在故障发生后仍保持数据库一致性及事务原子性的算法，主要分两部分
 - 在正常事务处理时采取措施，保证有足够的信息可用于故障恢复
 - 故障发生后采取措施，将数据库内容恢复到某个保证数据库一致性，事务原子性及持久性的状态。





目录

- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
- 检查点
- 缓冲技术





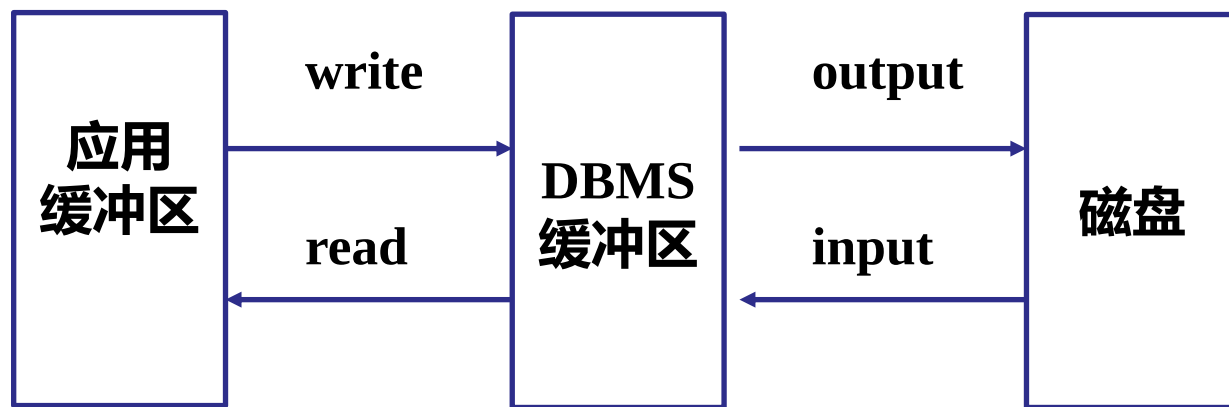
- DBMS进行故障恢复时会执行两种操作
 - Undo (撤销)
 - 撤销未完成事务对数据库的修改
 - Redo (重做)
 - 重做已提交事务对数据库的修改
- DBMS如何运用Redo和Undo取决于DBMS如何管理缓冲池 (buffer pool)
 - 只需undo?
 - 只需redo?
 - 既要undo, 又要redo?
 - 既不用undo, 也不用redo?





Undo Logging

- 基于Undo Logging的事务正常执行时的行为



应用程序决定

DBMS缓冲区
控制程序决定





缓冲池策略

初始 $A=1$, $B=2$

事务调度

T_1	T_2
BEGIN $r(A)$ $A := A + 1$ $w(A)$ ABORT	BEGIN $r(B)$ $B := B * 2$ $w(B)$ COMMIT

缓冲区

A=2	B=4	C=3

磁盘

A=?	B=?	C=3

- 是否强制在 T_2 提交时将 B 写回磁盘? $\leftarrow$ **FORCE策略**
- 是否允许在 T_1 提交前覆写磁盘 A 的值? $\leftarrow$ **STEAL策略**





缓冲池策略

- DBMS是否允许将来提交事务所做的修改写到磁盘并覆盖现有状态?

- STEAL: 允许
- NO-STEAL: 不允许

缓冲区

A=2	B=4	C=3

初始A=1, B=2

事务调度	
T_1	T_2
BEGIN $r(A)$ $A := A + 1$ $w(A)$	BEGIN $r(B)$ $B := B * 2$ $w(B)$ COMMIT
ABORT	

STEAL策略

磁盘

A=2	B=?

NO-STEAL策略

磁盘

A=1	B=?





缓冲池策略

- DBMS是否强制事务在提交前必须将其所做的修改全部写回磁盘?

- FORCE: 强制
- NO-FORCE: 不强制

缓冲区

A=2	B=4	C=3

初始A=1, B=2

事务调度	
T_1	T_2
BEGIN	
r(A)	
A := A + 1	
w(A)	
	BEGIN
	r(B)
	B := B * 2
	w(B)
	COMMIT
ABORT	

FORCE策略

磁盘

A=?	B=4

NO-FORCE策略

磁盘

A=?	B=2





- NO-STEAL+FORCE是最标准的方案

- NO-STEAL: 未提交的事务不可能将其修改写回磁盘 --» 无需undo
- FORCE: 已提交事务已将其修改写回了磁盘 --» 无需redo

初始A=1, B=2

事务调度

T_1	T_2
BEGIN	
r(A)	
$A := A + 1$	
w(A)	BEGIN
	r(B)
	$B := B * 2$
	w(B)
	COMMIT
ABORT	

缓冲区

A=2	B=4	C=3

磁盘

A=1	B=4	C=3

- 优点: 简单
- 缺点: 效率低



缓冲池策略

- **FORCE策略**: 主要关注commit后缓冲区修改和磁盘的关系。如果commit了, 必须写回磁盘 (每个事务结束都会触发I/O, I/O效率低)
- **NO-STEAL策略**: 主要关注对commit前缓冲区修改和磁盘的关系。事务没有结束, 缓冲区内容不能写回磁盘 (缓冲池一直占用, 使用效率低)

缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高





目录

- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
- 检查点
- 缓冲技术





预写式日志

- 预写式日志 (Write-Ahead Log, WAL) : DBMS在数据文件之外维护一个日志文件, 用于记录事务对数据库的修改
 - 假定日志文件存储在稳定存储器上
 - 日志记录包含undo和redo时所需要的信息
- DBMS在将修改过的对象写到磁盘之前, 必须先将修改此对象的日志记录刷写到磁盘





- 当事务 T_i 启动时，向日志中写入记录 **<tid, BEGIN>**
 - tid: T_i 的ID (txn ID)
- 当 T_i 提交时，向日志中写入记录 **<tid, COMMIT>**
 - 在DBMS向应用程序返回确认消息之前，必须保证 T_i 的所有日志记录都已刷写到磁盘
 - <tid, COMMIT> 写入磁盘才代表事务提交
- 当 T_i 修改对象A时，向日志中写入记录 **<tid, oid, before, after>**
 - oid: A的ID (object ID)
 - before: A修改前的值 (undo时用)
 - after: A修改后的值 (redo时用)





基于WAL的故障恢复

- 事务正常执行时的行为
 - 记录日志
 - 按照缓冲区策略将修改过的对象写到磁盘
- 故障恢复时的行为
 - 根据日志和缓冲池策略，对事务进行undo或redo





事务分类

- 根据日志将事务分为3类
- 已提交事务(committed txn)
 - 既有<T, BEGIN> , 又有<T, COMMIT>
- 不完整事务(incomplete txn)
 - 只有<T, BEGIN> , 而没有<T, COMMIT>
- 已中止事务(aborted txn)
 - 既有<T, BEGIN> , 又有<T, ABORT>
 - 在事务正常执行和故障恢复过程中, 如果T所做的修改已全部撤销, 则将日志记录<T, ABORT>写到日志
 - 已中止事务相当于从未执行过, 故不需要undo, 更不需要redo





故障恢复时的行为分类

- 已提交事务
 - 如果一个已提交事务的修改已经全部写到磁盘，则无需redo
 - 否则，需要redo
- 不完整事务
 - 如果一个不完整事务的任何修改都未写到磁盘，则无需undo
 - 否则，需要undo
- 缓冲池策略决定了上述行为





WAL协议分类

- 根据缓冲池策略的不同，可以实现三类WAL协议
 - Undo Logging: WAL + STEAL + FORCE
 - Redo Logging: WAL + NO-STEAL + NO-FORCE
 - Redo+Undo Logging: WAL + STEAL + NO-FORCE





- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
 - Undo Logging
 - Redo Logging
 - Redo+Undo Logging
- 检查点
- 缓冲技术





Undo Logging

- Undo Logging: 提交前缓冲区随时可以写回磁盘，提交时强制写回磁盘

$$\text{Undo Logging} = \text{WAL} + \text{STEAL} + \text{FORCE}$$

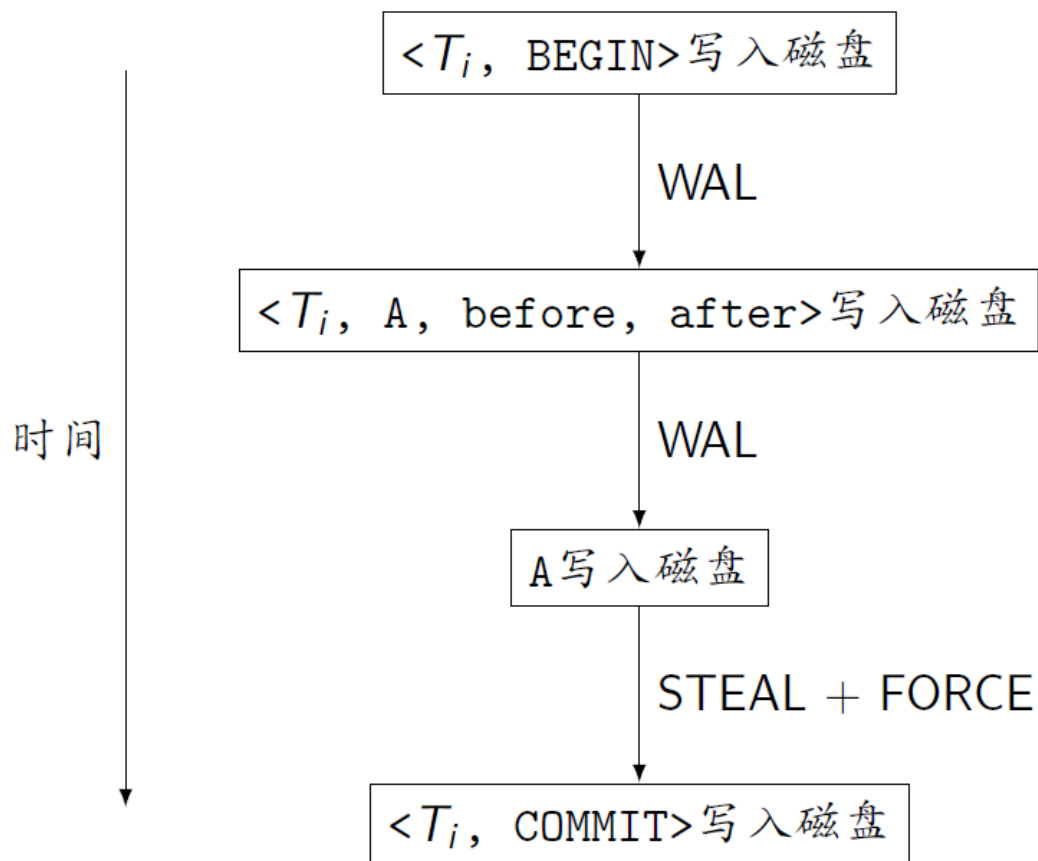
缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高





Undo Logging

- 基于Undo Logging的事务正常执行时的行为





Undo Logging

- 基于Undo Logging的事务正常执行时的行为

Example (Undo Logging)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
11	COMMIT						<T, COMMIT>
12	FLUSH LOG						





Undo Logging

- 基于Undo Logging的故障恢复
 - 已提交事务(Committed Txn):不需要恢复
 - FORCE=>已提交事务所做的修改已全部写入磁盘
 - 不完整事务(Incomplete Txn):全部undo
 - STEAL=>不完整事务所做的一部分修改可能已经写入磁盘





- 基于Undo Logging的故障恢复方法

- 从后(最后一条记录)向前(第一条记录)扫描日志
- 根据每条日志记录的类型执行相应的动作
 - $\langle T, \text{COMMIT} \rangle$: 将T记录为已提交事务(无需redo)
 - $\langle T, \text{ABORT} \rangle$: 将T记录为已中止事务(无需undo)
 - $\langle T, A, \text{before}, \text{after} \rangle$: 如果T是不完整事务, 则将磁盘上A的值恢复为before
 - $\langle T, \text{BEGIN} \rangle$: T恢复完毕; 如果T是不完整事务, 则向日志中写入 $\langle T, \text{ABORT} \rangle$ (今后故障恢复时无需再undo)





Undo Logging

• 基于Undo Logging的故障恢复

Example (基于Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
11	COMMIT						<T, COMMIT>
12	FLUSH LOG						
	Crash!						





Undo Logging

• 基于Undo Logging的故障恢复

Example (基于Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
	Crash!						





Undo Logging

• 基于Undo Logging的故障恢复

Example (基于Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							$\langle T, \text{BEGIN} \rangle$
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	$\langle T, A, 8, 16 \rangle$
5	READ(B, t)	8	16	8	8	8	
	Crash!						





- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
 - Undo Logging
 - Redo Logging
 - Redo+Undo Logging
- 检查点
- 缓冲技术





Redo Logging

- Redo Logging: 提交前缓冲区不能写回磁盘，提交后不是强制立即写回磁盘

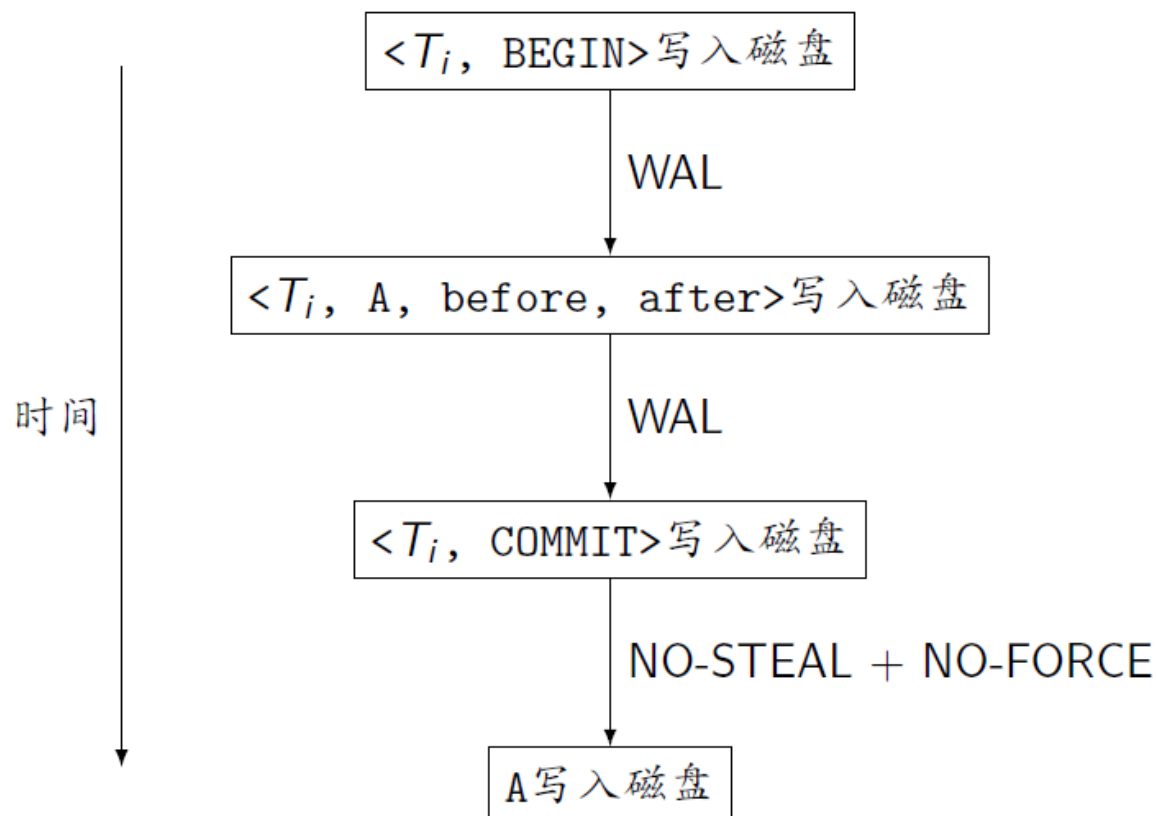
Redo Logging = WAL + NO-STEAL + NO-FORCE

缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高





• 事务正常执行时的行为





Redo Logging

- 基于Redo Logging的事务正常执行时的行为

Example (Redo Logging)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
10	OUTPUT(A)	16	16	16	16	8	
11	OUTPUT(B)	16	16	16	16	16	





- 基于Redo Logging的故障恢复

- 已提交事务(Committed Txn):全部redo
 - NO-FORCE=>已提交事务所做的修改可能尚未全部写入磁盘
- 不完整事务(Incomplete Txn):不需要恢复
 - NO-STEAL=>不完整事务所做的任何修改都未写入磁盘





• 基于Redo Logging的故障恢复方法

- 从前(第一条记录)向后(最后一条记录)扫描日志两遍
- 第1遍扫描:记录已提交事务和已中止事务
 - <T, COMMIT>:将T记录为已提交事务(需要redo)
 - <T, ABORT>:将T记录为已中止事务(无需undo)
- 第2遍扫描:根据每条日志记录的类型执行相应的动作
 - <T, A, before, after>:如果T是已提交事务, 则将磁盘上A的值覆写为after
 - <T, BEGIN>:如果T是不完整事务, 则向日志中写入<T, ABORT>





Redo Logging

• 基于Redo Logging的故障恢复

Example (基于Redo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
10	OUTPUT(A)	16	16	16	16	8	
11	OUTPUT(B)	16	16	16	16	16	
	Crash!						





Redo Logging

• 基于Redo Logging的故障恢复

Example (基于Redo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
	Crash!						





- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
 - Undo Logging
 - Redo Logging
 - Redo+Undo Logging
- 检查点
- 缓冲技术





Redo Logging

- Redo+Undo Logging: 提交前缓冲区可以写回磁盘，提交后不是强制立即写回磁盘

Redo + Undo Logging = WAL + STEAL + NO-FORCE

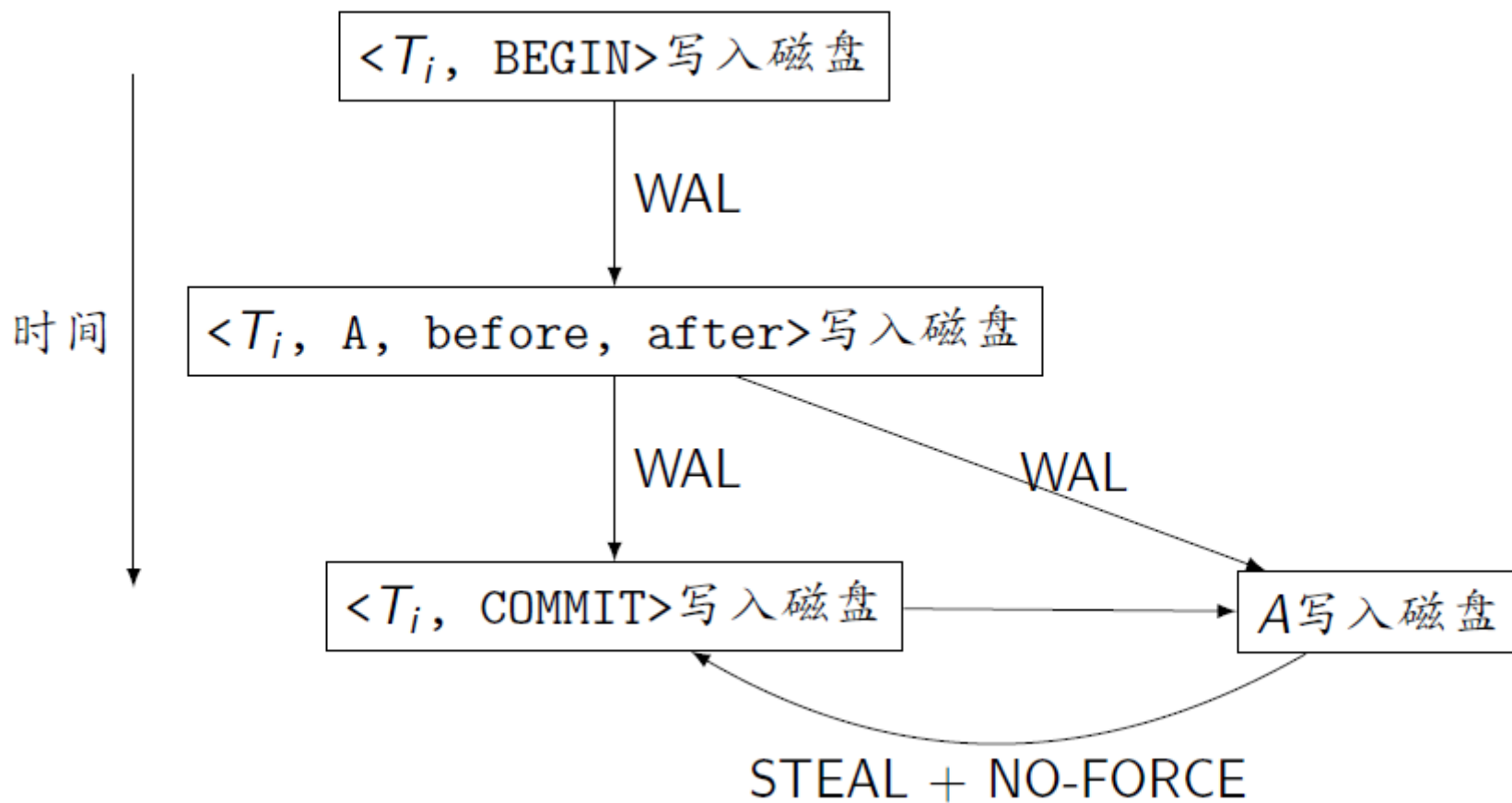
缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高





Redo+Undo Logging

- 基于Redo+Undo Logging的事务正常执行时的行为





Redo+Undo Logging

- 基于Redo+Undo Logging的事务正常执行时的行为

Example (Redo+Undo Logging)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
9	COMMIT						<T, COMMIT>
10	FLUSH LOG						
11	OUTPUT(B)	16	16	16	16	16	





Redo+Undo Logging

- 基于Redo+Undo Logging的故障恢复

- 已提交事务(Committed Txn):全部redo
 - NO-FORCE=>已提交事务所做的修改可能尚未全部写入磁盘
- 不完整事务(Incomplete Txn):全部undo
 - STEAL=>不完整事务所做的一部分修改可能已经写入磁盘





Redo+Undo Logging

- 基于Redo+Undo Logging的故障恢复方法

- Redo阶段:redo已提交事务
 - 与基于Redo Logging的故障恢复方法相同
- Undo阶段:undo不完整事务
 - 与基于Undo Logging的故障恢复方法相同





Redo+Undo Logging

• 基于Redo+Undo Logging的故障恢复

Example (基于Redo+Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
9	COMMIT						<T, COMMIT>
10	FLUSH LOG						
11	OUTPUT(B)	16	16	16	16	16	
	Crash!						





Redo+Undo Logging

- 基于Redo+Undo Logging的故障恢复

Example (基于Redo+Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
	Crash!						





缓冲池策略的比较

- 几乎所有的DBMS都采用STEAL+NO-FORCE

运行时效率

	FORCE	NO-FORCE
STEAL	—	Fastest
NO-STEAL	Slowest	—

故障恢复效率

	FORCE	NO-FORCE
STEAL	—	Slowest
NO-STEAL	Fastest	—





- 几乎所有的DBMS都采用STEAL+NO-FORCE
 - 每条日志记录单独刷写(flush)到磁盘的I/O开销太大
 - 在内存中设置日志缓冲区(log buffer), 将日志记录写到日志缓冲区, 然后成批刷写到日志文件
 - 日志缓冲区满时刷写
 - 定时刷写





- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
- 检查点
- 缓冲技术





- 日志永远在变大
- 故障恢复时需要扫描日志，恢复时间越来越长

Example (WAL)

<T₁, BEGIN>
<T₁, A, 5, 15>
<T₂, BEGIN>
<T₂, B, 10, 20>
<T₂, C, 15, 25>
<T₂, COMMIT>
<T₃, BEGIN>
<T₁, D, 20, 30>
<T₃, E, 25, 35>
<T₁, COMMIT>
<T₃, F, 30, 40>

如果使用Undo Logging，则扫描到这里即可





- DBMS定期设置检查点(checkpoint)
 - 将日志刷写到磁盘
 - 根据缓冲池策略，将脏页(dirty page)写到磁盘
 - 故障恢复时只需扫描到最新的检查点





模糊检查点(Fuzzy Checkpoints)

- 检查点开始:向日志中写入 $\langle \text{BEGIN CHECKPOINT}(T_1, T_2, \dots, T_n) \rangle$
 - $T_1, T_2, \dots, T_n$ 是检查点开始时的活跃事务(active txn)
 - 活跃事务是尚未提交或中止的事务
- 检查点中间:根据缓冲池策略,将脏页(dirty page)写到磁盘
 - 如果采用STEAL,则将全部脏页写到磁盘
 - 否则,只将已提交事务所做的修改写到磁盘
- 检查点结束:向日志中写入 $\langle \text{END CHECKPOINT} \rangle$,并将日志刷写到磁盘
 - 如果采用NO-FORCE,则写完全部脏页后即可结束检查点
 - 否则,在 $T_1, T_2, \dots, T_n$ 全部提交后,才能结束检查点





涉及检查点的故障恢复

- 缓冲池策略:STEAL+ NO-FORCE(自行思考其它策略下的恢复方法)
- Redo阶段:redo已提交事务
 - 从前向后扫描日志
 - 从哪条日志记录开始?
- Undo阶段:undo不完整事务
 - 从后向前扫描日志
 - 到哪条日志记录为止?

<BEGIN CHECKPOINT ($T_1, T_2, \dots, T_n$)>

...

<END CHECKPOINT>





涉及检查点的故障恢复

- Redo阶段

- 日志中最新的完整检查点

$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$

...

$\langle \text{END CHECKPOINT} \rangle$

- 需要redo的最早的事务一定属于 $\{T_1, T_2, \dots, T_n\}$
- 从日志记录 $\langle \text{BEGIN CHECKPOINT}(T_1, T_2, \dots, T_n) \rangle$ 开始向后扫描日志
 - 不需要从最早的 $\langle T_i, \text{BEGIN} \rangle$ 开始扫描





涉及检查点的故障恢复

• Redo阶段

证明 I

需要redo的最早的事务 T 一定属于 $\{T_1, T_2, \dots, T_n\}$

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle T, \text{COMMIT} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \notin \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的修改已全部写到磁盘，无需redo





涉及检查点的故障恢复

• Redo阶段

证明 II

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle T, \text{COMMIT} \rangle$	
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的修改已全部写到磁盘，无需redo





涉及检查点的故障恢复

• Redo阶段

证明 III

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	
...	
$\langle T, \text{COMMIT} \rangle$	T 所做的修改未必全部 写到磁盘，必须redo





涉及检查点的故障恢复

- Undo阶段

- 日志中最新的完整检查点

$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$

...

$\langle \text{END CHECKPOINT} \rangle$

- 需要undo的最早的事务一定属于 $\{T_1, T_2, \dots, T_n\}$
- 扫描到 $T_1, T_2, \dots, T_n$ 中最早的事务 T_i 的日志记录 $\langle T_i, \text{BEGIN} \rangle$ 为止





涉及检查点的故障恢复

• Undo阶段

证明

需要undo的最早的事务 T 一定属于 $\{T_1, T_2, \dots, T_n\}$

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的部分修改可能已写到磁盘，必须undo





涉及检查点的故障恢复

- Redo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	$C \leftarrow 25$
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	





涉及检查点的故障恢复

- Redo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	$C \leftarrow 25$
$\langle T_3, D, 20, 30 \rangle$	T_3 无需redo
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	





涉及检查点的故障恢复

- Redo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	$C \leftarrow 25$
$\langle T_3, D, 20, 30 \rangle$	T_3 无需redo
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 redo完毕





涉及检查点的故障恢复

• Undo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo





涉及检查点的故障恢复

• Undo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo





涉及检查点的故障恢复

• Undo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	T_2 无需undo
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo





涉及检查点的故障恢复

• Undo (STEAL+NO-FORCE)

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	T_3 undo完毕, 写 $\langle T_3, \text{ABORT} \rangle$
$\langle T_3, B, 10, 20 \rangle$	$B \leftarrow 10$
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	T_2 无需undo
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo





目录

- 数据库恢复的必要性
- 缓冲池策略
- 基于日志的数据库恢复技术
- 检查点
- 缓冲技术





缓冲技术

- 日志缓冲技术
- 数据库缓冲技术





- 日志缓冲技术

- 通常一个日志记录远远小于存储器的读写单位。这样，经常向存储器写单个日志记录将导致很大的系统开销。
- 对日志记录的读写操作使用缓冲技术，即在主存中设立缓冲区，其大小等于永久存储器的读写单位，被写的日志记录先存储到缓冲区，缓冲区满之后再一起永久写入存储器。





• 日志缓冲技术

- 缓冲区中的日志记录在系统发生故障时会丢失。为了保证事务的原子性，需要在数据库恢复协议上增加如下的规则：

- (1) 任何事务T必须在日志记录 $\langle T, \text{commit} \rangle$ 已经永久写入存储器以后才可以进入提交状态。
- (2) 任何事务T的非 $\langle T, \text{commit} \rangle$ 型日志记录必须在日志记录 $\langle T, \text{commit} \rangle$ 之前永久写入存储器。
- (3) 主存缓冲区中的数据库数据必须在所有与这些数据有关的日志记录被永久写入存储器之后，才可以永久写入存储器中的数据库。





• 数据库缓冲技术

- 当系统要把数据块 B_2 读入主存并覆盖数据块 B_1 时，它必须按如下方式进行：

- IF B_1 已经被修改
- THEN
 - 输出有关 B_1 中数据的所有日志永久地记录到存储器；
 - 输出 B_1 到磁盘；
 - 从磁盘输入 B_2 到内存缓冲区；
- ENDIF。





• 本章重点

- 掌握故障的分类
- 掌握使用日志的数据库恢复技术





Game Over

Wish you enjoy the
course

